claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-a5a7cb166dc148490.jsonl`
- SHA-256: `e0d408679dec0fe213f65b30fdcd7c2c3a7093cccfffb562c27935dc51a4355b`
- Source modified: `2026-06-19T15:50:18+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_ac0cbb29-6bf`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`

Transcript

1. user (2026-06-19T15:46:34.349Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.
HARD RULES:
- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.
Working dir is the repo root; use Read/Grep/Glob freely.

YOUR SLICE ? codemap:
docs/CODE_MAP.md + docs/README.md doc-index module-structure staleness vs CURRENT code. Recent refactors moved/split modules: #226 backend/eval/ablation_pdf.py, #227 yolo_hybrid phase helpers, #228 backend/api/uploads.py router, #230 motion detection/populate halves, #231 _build_preset in rally_seg_eval, #232 db _execute_write, #233 backend/eval/golden_real_fixtures.py, #234 backend/api/job_worker.py, #235 shared golden-feature loader, #215 _is_fusion_schema SSOT in backend/eval/experiment.py, #220 long-rally lens (--min-duration/--stratify in rally_seg_eval + stitching.min_rally_duration + tools/generate_highlights --min-rally-duration). For each file path / symbol / location CODE_MAP or README claims, Glob/Grep to confirm it exists where stated; report wrong/missing paths and moved symbols. Also flag new modules absent from the map if the map purports to be complete.

Return EVERY code-contradicting staleness item you can verify in your slice. Exhaustive within the slice; facts only.

2. assistant (2026-06-19T15:46:36.940Z)

I'll audit the codebase for staleness in `docs/CODE_MAP.md` and `docs/README.md` against the current code. Let me start by reading both documents and the recent refactor changes.

3. user (2026-06-19T15:46:38.262Z)

1   - ROORA_LABELING_GUIDELINES.md ? Latest (v8) vendor labeling guidelines mirrored from the collector's `docs/annotation/`, with the fusion-driven court-calibration / singles-doubles additions marked `? ADDED (fusion)`.
2   - RALLY_QUALITY_RESEARCH.md ? **? Foundation doc** for closing the local-vs-Gemini rally-detection gap: the measured gap + the code-grounded over-merge diagnosis ("shuttle moving ? rally"), an inventory of the eval/experiment harness + its gaps,

the strengthened evaluation framework (segmental Edit/F1@k that make over-segmentation first-class, golden-corpus + active learning, Gemini-agreement as a *shadow* signal), an **adversarially-verified, cited external-research synthesis** (TAL/TAS boundary heads, racket-sports rally-state cues, small-data, eval rigor), a **prioritized cheap?durable roadmap with falsifiable success criteria**, and (§7) a **tracked project plan** ? work-item checklist, sequencing, the single quality **number** attributed to the effort, and a **definition of done** (project completes when all tiers land + the number is recorded). The plan that the QUALITY_ITERATIONS.md log executes against; ties together EXPERIMENT_HARNESS / MULTI_SIGNAL_FUSION_PLAN / ANALYZERS_AND_RECONCILERS / OWNED_MODEL_IMPLEMENTATION_PLAN.
3 - RALLY_DETECTION_GAP_CLOSURE.md ? **Tracked execution doc (2026-06-18)** for closing the remaining rally-detection accuracy gap **per video, before launch**, driven by the growing golden corpus. First ground-truth clip (GX010141) reframed it: boundaries are tight (R2/R3/R4 holds) ? the gap is **detection**: (G1) recall (local & Gemini both miss ~half on hard clips, and *different* rallies ? complementary), (G2) precision (local over-fires), (G3) **rally-END over-extension on the quick-return-after-point** (shuttle still moving ? needs a *player-state* signal). Levers (owner-gated): player-kinematics + shuttle-in-hand / sustained-invisibility end-rule with reverse-timeline backtrack, `rally_gate` Gate A wiring, complementary fusion. **Peer** of `RALLY_QUALITY_RESEARCH.md` (execution tracker, not research); §7 is the source of truth.
4 - RALLY_DETECTION_QUALITY_REPORT.md ? **Technical-paper checkpoint (2026-06-17)** of rally-detection quality: the honest measured state (over-segmentation milestone tolerance-F1 0.091?0.323 at n=11, p=0.002; merge-rate 0.694?0.150; first real-footage ground-truth at-par with START at Gemini parity), the task-faithful R2 metric, a detailed Gemini comparison, next steps + expected wins, areas yet to be explored, and a publishability / path-to-submission verdict. Renders to a polished PDF; the externally-shareable "toward a paper" summary of `RALLY_QUALITY_RESEARCH.md` + `QUALITY_ITERATIONS.md`.
5 - EVALUATION.md ? How we measure and improve quality.
6 - EXPERIMENT_HARNESS.md ? Design (P1 SHIPPED) for A/B + shadow + SxS experimentation on rally-detection variants with **zero production regression** ? experiments are decoupled from deploy (new cues default OFF, promote only on measured LOVO lift, rollback = flip a flag). Composes the existing `backend/eval/` machinery (`calibration`, `regression`, `metrics`); implemented in `backend/eval/experiment.py`.
7 - QUALITY_ITERATIONS.md ? **Living log** of rally-detection quality iterations (hypothesis ? what shipped ? measured/expected effect ? lesson). The reverse-chronological record; terminal-state snapshots get archived under `archives/quality-iterations/`.
8 - ANALYZERS_AND_RECONCILERS.md ? Design (owner-gated, NOT started): a **signal-fusion framework** for rally detection ? producers emit confidence-scored **signals** at three temporal **scopes** (frame detectors / window analyzers / session analyzers), a learned **scorer** weights them, and an auditable **report-first reconciliation** pass lints the decision against the evidence. The spine: **no special-casing in the decision path** (signals?scorer, never `if/else`). Worked examples: a service-fault analyzer, a warm-up/practice span detector, and consuming a per-frame shuttle-state detector. A peer to `EXPERIMENT_HARNESS.md` and `MULTI_SIGNAL_FUSION_PLAN.md` (ADR-007 modular rally layer).
9 - PERSONALIZATION_PLAN.md ? Design (owner-adopted **hybrid**; Phase-0 landing) for the **per-user personalization feature on a generalization-first baseline**: a thin per-user *tuning* layer gated by the honest small-N statistics (a **min-N promotion floor** + a **structural-guard exemption**, never a service gate) + a **contribution flywheel** (free-tier videos pool into the owner-trained baseline; *"a tool that gets better the more you help it"*). Records the **locked owner decisions** (identity, storage, `min_n` semantics, Gate-A default, free-tier pooling). Built so far: per-user promotion gate (#141) + held-out-USER learning curve (#142).
10
11 - GOLDEN_REGRESSION_FIXTURES.md ? **Design (owner-gated, NOT started):** video-free, non-attributable regression fixtures so a clean clone runs the rally-seg suite with **no video/GPU/DB**. Two layers ? the post-detector freeze + **dual gates** (characterization + quality-floor) reusing the `backend/eval/` stack, and the **SEALED-FIXTURES** privacy filter (de-attribution-at-source, tiered public/key-gated, shuttle-only public tier). Carries a cited **IN/US/EU legal memo** (derived-data IP/privacy), a threat model, and a **§7 progress tracker + definition-of-done**. Extends GOLDEN_DATA_SHARING.md.
12
13 ## Product / platform plans
14 - DESKTOP_APP_PLAN.md ? **Tracked scoping doc (2026-06-18, owner-

gated, `[design]` ? NOTHING built):** package the local pipeline as a **cross-platform desktop app** for non-technical players ? plug in the camera's USB/SD card, click once, get back **only the highlight reels**, fully on-device (no upload, no original-video bloat). The centerpiece is the **de-WSL native-compute port** (run WASB natively per-OS: Linux/Windows CUDA · macOS MPS/CPU ? drop WSL2), and its **make-or-break unknown**: is CPU acceptable on a typical *no-GPU* enthusiast laptop (WASB is ~3.4× real-time on a laptop 2070S)? §7 tracker = **P0 compute-feasibility spike ? P1 native detector ? P2 app shell ? P3 packaging/installers**. Realizes the **L0 fully-local** tier of VIDEO_LOCALITY_MODEL.md; is the **`LocalDesktop` ComputeBackend** of PLATFORM_ARCHITECTURE.md §3.2; the smaller/export-clean owned model (OWNED_MODEL_IMPLEMENTATION_PLAN.md) is one answer to the compute crux.
15
16   ## Practical / ongoing
17   - TRACKNET_WSL_SETUP.md
18   - Golden set docs (`GOLDEN_SET_*`)
19   - Evaluation harness and calibration drivers (in `backend/eval/`)
20   - GOLDEN_DATA_SHARING.md ? How the GPU box and CPU dev/agent sessions share golden artifacts (trajectory CSVs, `*_golden_features.json`) via a OneDrive folder; the GPU-extract ? CPU-litmus split. See **#175**.
21
22   ## Conventions & templates
23   - PROJECT_DOC_TEMPLATE.md ? Reusable skeleton for a **tracked project doc**: status header · decisions-locked · progress tracker (?/?/?) · definition-of-done · archived on completion. Standardizes the emergent pattern in `RALLY_QUALITY_RESEARCH.md` §7 and `GOLDEN_REGRESSION_FIXTURES.md`. Use for any substantial design/project work.
24
25   ## Historical documents
26   Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).
27
28   - CODE_AUDIT_AND_TEST_HARDENING.md ? **COMPLETED** (2026-06-14; T1?T5 + full loop + verification per #162). Living record of the June 2026 plumbing / contract / architectural audit, 626-test infrastructure, positive test plans for the 5 identified issues, multi-layer regression protocol (including ablation/experiment layer), and pre/post-change verification checklist. See `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` and `docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md` for the final archived state. (Top-level is now pointer stub per review; full content in archives/.)
29
30   See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md` for classification of small tech debt / starter projects.

4. user (2026-06-19T15:46:38.380Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-210 of 384 total (38759 tokens, cap 25000). Call Read with offset=211 limit=210 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1   # CODE_MAP.md
2
3   **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5   **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).
6
7   ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;

`match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`, `test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`, `test_import_guard`, `test_cvat_robustness`).

LEGEND

- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing **registry / extension point** (`?`) over a new top-level file.

You're adding?	Put it in?	Notes
Backend / runtime code	`backend/<subsystem>/`	`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree.
Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)	`backend/eval/`	Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not scripts; do NOT move them out.
Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code)	**`scripts/`**	e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`.
Ops / maintenance tool (not part of the served app ? cache GC, batch admin)	`tools/`	Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested.
Training code (model fitting)	`training/`	Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`.
A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.
A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean checkout); the no-regression gate reads it.
Committed regression fixture (video-free golden)	`eval_baselines/fixtures/<slug>/`	Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated).
Pipeline run outputs (reels, trajectories, DBs, frames)	`output/`	**Gitignored.** Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`.

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):

`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) ·

```
42      `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) ·
`run_eval.py`
43      (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44      `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45      New standalone run-drivers go in **`scripts/`**, not the root.
46
47      ---
48
49      ## 1. PIPELINE
50
51      ### 1A. Runtime: video file -> highlight reel
52      ```
53      typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54                                                  @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
55        -> video_id = MD5(whole file)             @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56        -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57             FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
58             [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59        -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
60        -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
61        -> seg = segmenter_registry.get(seg_name)(db, global_config)   ?
@backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
62        -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
63             [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
64        -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
65        -> select segment ids -> [(start,end)] (+ stitching padding)
66        -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
67             per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
68      ```
69      Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
70      ```
71      P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
72       -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted
via db.add_candidate_segment]
73       -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt)   ? provider_registry
74       -> P4 db.add_play_segment + db.link_candidate_to_segment
75      ```
76      Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
77      Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
78
79      ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
80      ```
81      WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
82       -> WasbRunner.healthcheck()  (gate; not ok -> return [])
```

@backend/pipeline/detectors/wasb_runner.py
83   -> WasbRunner.run_predict(video, out_dir)
84   stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
85   -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
86   -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
87   -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
88   ```
89   (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
90
91   ### 1C. Calibration / eval flow (NO pipeline run unless told)
92   ```
93   GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
94   DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage
? {candidates, final}
95   -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
96   -> harness.evaluate -> EvalReport -> report_to_dict
97   -> batch.aggregate (corpus micro/macro)   @backend/eval/batch.py
98   -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py
[CI gate]
99   Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
100   Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
101   Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
102   Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
103   Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
104   ```
105   Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@scripts/nightly_regression.py` (nightly golden-corpus rally-quality tripwire, exit
1=regression/4=stale), `@backend/eval/calibrate_wasb.py` +
`@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107   ---
108
109   ## 2. SUBSYSTEMS
110
111   ### 2.0 Orchestration & config
112   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
113   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
114   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::_get_executor` lazy module-global

`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115 - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in `@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`; `uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the startup config, and a lazy `import backend.main` avoids an import cycle) ? `uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight` `@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
116 - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
117 - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
118 - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
119 - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
120 - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
121 - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
122 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.
123 - `@scripts/nightly_regression.py::main` ? **nightly rally-quality tripwire** (the dual-eval-set discipline made automatic). Re-scores the golden corpus (`output/human_lovo_manifest.json`) under TWO configs ? `default` (SERVED, milestone knobs OFF) + `milestone` (R3 cap=6 + R4 inactivity trim) ? via `backend.eval.rally_seg_eval.evaluate` (CPU, seconds; CACHED trajectories, **no GPU/Gemini/WASB**), diffs headline metrics (`tol_f1`, tIoU `f1@0.5/0.25`, `merge_rate`) + the **per-video over-seg guard** (`split_count`/`merge_count` may not rise on ANY video) vs frozen `eval_baselines/nightly_baseline.json`. Pure core (`summarize_run`/`compare_config`/`compare_all`/`format_report`) is unit-tested; thin I/O shell scores + writes `output/nightly_regression/<date>.{md,json}`. Exit **0=pass, 1=regression, 2=missing corpus, 3=no baseline, 4=stale (corpus/config changed ? re-snapshot)**. `--update-baseline` snapshots HEAD. **Default-ADDITIVE ? does NOT change the promotion gate.** Wired as a local Windows Scheduled Task via `scripts/nightly_regression.ps1` (wrapper) + `scripts/register_nightly_task.ps1` (a cloud agent can't ? the cached trajectories live outside the repo). The future **RAW eval set** (currently-bad videos) plugs in as a third config. See `docs/R2_EVAL_METRIC_DESIGN.md` + `memory/eval-dual-set-regression`.
124

125   - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `StitchingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 7. Use `from backend.config import load_config, AppConfig`.
126   - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
127   - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
128   - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
129   - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
130   - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal of 5 sports. `::Config = AppConfig` alias.
131   - `@backend/config/loader.py` ? load/save with defaults-merge.
132   - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
133   - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
134   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by scripts/doctor.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
135   - `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
136   - `@scripts/doctor.py` (formerly root `setup.py`) ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer --setup`.
137
138   ### 2.1 Segmenters (under pipeline/segmenters) ?
139   - `@backend/pipeline/segmenters/base.py`
140   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
141   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
142   - `@backend/pipeline/segmenters/__init__.py` ? imports all 6

(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
143   - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ? SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider handoff -> P4 DB/link; single `::_shot_count_from` (old wasb/tracknet shot_count drift gone). NOT registered itself. ? Subclass contract: `family` (config section `indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir, `<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description` properties, `_build_runner(idx_cfg) -> (DetectorRunner, detection_params extras)`. **Two fusion seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test): `::_enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4 motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate) **and `::_select_for_handoff(windows,window_stats,idx_cfg)`** (base = all indices). ? shared knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass; needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by `tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the base-hook-identity tests ? parametrized over both twins).
144   - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name `'fusion_hybrid'`, **default-OFF** multi-signal fusion P2) ? subclasses the engine; `family='fusion'`, `display_label='Fusion'`. `::_build_runner` selects the substrate via `indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config. `::_enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via `rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features (`::_person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-video frame range ? `fusion_features.compute_person_features`). `::_select_for_handoff` overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`, person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads `idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3 cleanup). Tests: `tests/test_fusion_hybrid.py`.
145   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras = `weights_path` only.
146   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires `TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.
147   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to `detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to `self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the `detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_*` config keys are deprecated aliases still honored; still mypy-quarantined for pre-existing orchestration debt (the torchvision code that caused it left).
148   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. Chunks are RE-ENCODED + downscaled to `indexing.ai_chunk_scale_width` (default 1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's `MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
149   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
150
151   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)

- `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
- `@backend/pipeline/detectors/tracknet_runner.py`
  - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
  - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
  - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
- `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` now shared via `detectors/base.py::WslCommandMixin` (2026-06-11 dedupe). Both runners now satisfy the DetectorRunner ABC for future swappability. **Cache hygiene (#109):** on a SUCCESSFUL run both runners delete their WSL intermediates ? WASB the frame cache + staged video, TrackNet the staged video ? gated by `indexing.{wasb,tracknet}.keep_frames` (default false; on FAILURE kept so a re-run resumes). Pre-run `min_free_gb` disk-guard warning. Shared helpers `WslCommandMixin::_wsl_rm_rf`/`_wsl_free_gb`.
- `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
- `@backend/pipeline/detectors/person_detector.py::PersonDetector` ? the **PERSON rally cue** (torchvision detector + supervision ByteTrack + velocity windowing), extracted from yolo_hybrid (fusion P1; type-clean, NOT quarantined). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. `::lazy_load_model` (lazy `torchvision` import so module/tests load without it; `self.model` typed `Any`), `::make_tracker` (per-chunk ByteTrack ? IDs chunk-local), `::detect_and_track` (one frame -> confident persons w/ track IDs; BGR->RGB, COCO person filter, ByteTrack association), `::extract_action_windows_from_chunk` (chunk -> high-motion windows w/ `peak/mean_velocity`, `active_frame_count`, `track_id_count` ? the stats that feed candidate `stats` AND the future learned-fusion person features). ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`. Consumed by yolo_hybrid AND the fusion segmenter (P2 reuses the SAME producer over shuttle-seeded windows via `::detections_per_frame` ? add-a-producer, not a refactor). BSD torchvision + MIT ByteTrack.
  `::detections_per_frame(video_path,start_frame,end_frame,frame_skip)` (fusion P2) reads the ORIGINAL video by ABSOLUTE frame range (seek+sequential) ? `{frame_idx: [(track_id,bbox)]}` + `fps/frame_width/frame_height`, so indices align 1:1 with the WASB trajectory `.frame` (no clip re-timing). ? cv2 frame-seek accuracy varies by codec (±a few frames on real footage).
- `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue feature math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame `{frame_idx:[(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon` + `net=(axis,pos)`. `::point_in_polygon` (ray-cast), `::person_in_court` (foot-point; None polygon=count-all; bad dims=False), `::in_court_counts`, `::players_in_court` (median), `::in_court_ratio`, `::mean_player_motion` (per-track centre displacement; x by width, y by height), `::two_sided_motion` (both net-halves occupied), `::shuttle_player_colocation` (shuttle inside a person box ±`pad_frac` halo over aligned visible frames ? **the held-shuttle discriminator**), `::compute_person_features` (aggregator ? the 5 `::PERSON_FEATURE_NAMES`). All

scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
161   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds;
`estimate=` param lets a caller scoring many windows pass a precomputed `(axis,net,span)` to
skip per-window re-estimation ? used by FusionSegmenter).
`::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is
`(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments / fusion_hybrid.
162
163   ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
164   - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
165   - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
166   - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. §
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>~0.1` = overfit alarm.
167   - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
168   - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. §
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
169   - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
170   - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
171   - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
172   - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
173   - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
174   - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid

vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ? `window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ? P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate + metrics.
175 - `@backend/features/rally_seq.py` ? **THE single source of truth for the Gen-0 rally_seq per-frame feature schema** (ADR-008): training (`training/`) AND the serving/eval path import the SAME `build_frame_arrays`/`features` so train==serve by construction. Pure numpy/scipy (no torch/cv2). `::FEATURE_SCHEMA_VERSION` (=1; bump on ANY semantic change ? artifacts pin it, loader hard-fails on mismatch), `::FEAT_NAMES` (the 6: `vis_rate,spd_mean,spd_max,cross_rate,x_std,y_std`), `::FEATURE_SCHEMA` (`win_sec`/`stride_sec`/`fps_semantics`/`occlusion_rule`), `::build_frame_arrays(points,frame_width,fps)` (per-frame vis/x/y + per-visible-frame speed & net-crossings; speed normalized per-SECOND `*fps`, NOT per-frame ? the 2026-06-10 fps-skew fix; skips speed/crossings across occlusion gaps > `max(2,round(0.25*fps))`), `::features(arr,fps,win_sec,stride_sec)` (scipy `uniform/maximum_filter1d` windowed -> `(X[n,6], times)`). ? re-exported by `@backend/eval/rally_seq_proto.py`.
176 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter = owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean, spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. ? `build_frame_arrays`/`features` (+`FEAT_NAMES`) are NOT defined here ? they are **re-exported** (`noqa F401`) from `@backend/features/rally_seq.py` (ADR-008 single-sourcing; speed per-SECOND + occlusion-gap-guarded since the 2026-06-10 sweep). Locally-owned: `::labels_for`, `::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based with **midrank ties**, scipy.stats.rankdata), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n=6, linear, threshold-transfer unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). **n=6 LOVO: learned 0.626 vs heuristic 0.480** (learned rose from 0.583 after the fps-normalization fix). Manifest `output/human_lovo_manifest.json` (n=6; trajectory paths now under `Annotation Setup/Collect/Trajectories/`) shared w/ calibrate_local.
177 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court isolation** (spatial gate to the prominent court, needs real court-region detection). Academy = multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
178 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal), `::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.
179 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; eval/tuning, NOT the live pipeline). `::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (? FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`, `::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises SystemExit); imports `BASELINE` from calibrate_wasb + `TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
180 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`, `::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored output/), `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`; "file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle), `::save_baseline`/`::load_baselines`, `::gt_hash` (label-set fingerprint, reused by experiment.py), `::RegressionResult`. ? imports `annotation_registry` from **`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop (F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI `run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
181 - `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin "experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes metrics/calibration/classifier/training/regression ? NO new scoring math). `::Variant`(segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate B],classifier_model,feature_names,threshold,merge_gap; `to_dict/from_dict/spec_hash/is_learned`), `::CONTROL` (pinned zero-gate baseline). `::VideoCandidates`(name,intervals,features{col-major},gts), `::predict` (gate?learned-

filter?`gemini_refine.merge_overlaps`), `::evaluate_variant` (static / pinned-model / **honest-LOVO** train-on-others ? mirrors distill_local), `::compare` (labeled **B?A** deltas + `label_set_hash`), `::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via `match_intervals`, no GT ? the owner's "SxS on new videos"), `::record_experiment` (? `eval_baselines/experiment_leaderboard.json`), `::load_video_candidates` (DB bridge via `query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg invariant (tested): all cues OFF ? byte-identical to CONTROL.**
182
183 - `@backend/eval/golden_fixtures.py` ? **video-free regression harness** (`docs/GOLDEN_REGRESSION_FIXTURES.md`). `::replay_fixture` = `tracknet_runner.parse_trajectory_csv` ? `distill_local.build_candidates`(PINNED preset) ? `experiment.VideoCandidates`(5 shuttle cols only) ? `experiment.predict(CONTROL)` ? `metrics.score`; **CONTROL / pure-stdlib ? no numpy on the replay path** (cross-OS stable; the learned/numpy path is deliberately excluded). `::freeze_synthetic` (3 committed synthetic Tier-0 fixtures), `::freeze_real_fixture` (P1/M2: `RefusalError` unless `owned`+`minors_free`+non-blank `license`; opaque random `fx_<hex>` slug, metric-invariant time-origin reset, STRICT `gt_loader`, positive no-`FORBIDDEN_KEYS`/MD5/source-path scan), `::refresh_snapshot`, `::compare_expected` (exact-int / approx-1e-6, NEVER byte-compare), `::load_manifest`/`::load_fixture` (schema_version dispatch). CLI `--freeze-synthetic`/`--freeze-real`/`--refresh-snapshot`/`--check`. Tests: `tests/test_golden_fixtures.py` (M1/M3 characterization) + `tests/test_golden_real_fixtures.py` (P1/M2). CI: the `golden-crossos` job (ubuntu/windows/macos) runs `--check` + the M1 test. ? synthetic-tier numbers = plumbing correctness, NOT field quality.
184
185 ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
186 - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength), `::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`; `k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted (PR #35).
187 - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion). `::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally), `::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses `hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
188
189 ### 2.4 Stitcher / tools / utils
190 - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw, segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time` (mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads `config.json stitching.{begin_padding,end_padding}` (typed default 0.5/0.5); temp clips in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so final concat can stream-copy ? change a preset and concat may break. ? optional **branding watermark** (`stitching.watermark`, default-on): `::_watermark_filter` builds a `-vf` (drawtext via `textfile=` + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel, and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a `WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0 `::_BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without fontconfig; `font_path` overrides.
191 - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows), `::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
192 - `@tools/generate_highlights.py` ? **the batch highlight RUNNER** (the clean, orchestrated way to do "make reels for these videos into this folder"). `::generate_highlights_batch(video_paths, out_dir, *, segmenter, local_work_dir, db_path, force)` + `::_atomic_publish` (copy?`os.replace`; never deletes siblings) + `::_resolve_videos`. **Guarantees:** generates LOCALLY then publishes atomically into the dest (Drive-sync-safe ? never generate inside a Google-Drive folder); **NEVER bulk-deletes** the dest; per-video

`clear_video_data` (idempotent ? no dup-rally accumulation; pairs with the compiler overlap-merge); resumable (skip published reels unless `--force`); per-video `<stem> - run.log` + `_batch_run.log` published additively. CLI: `python -m tools.generate_highlights --videos-dir <dir> --out <dir> --segmenter gemini [--force]` or repeated `--video`. ? WSL detectors (wasb/tracknet) localize the source first (Drive invisible to /mnt). **Use THIS for batch reels ? never hand-roll a loop that writes into Drive or rm's the folder** (that lost finished reels + run.logs once).
193   - `@tools/cleanup_caches.py` ? **top-level** `tools/` ops tool (NOT backend runtime). Dry-run-first cache GC. Pure `::classify`(name,is_dir,location)/`::plan_deletions` (unit-tested = the destructive decision path) + I/O `::scan_wsl` (du+find; ? arg-globbing NOT a `for` loop ? the login shell empties loop vars) / `::scan_windows` / `::_delete`. CLI: `python -m tools.cleanup_caches` (report) · `--apply` / `--keep-video <id|stem>` / `--older-than N` / `--include-wasbcache` / `--summary` (hook nudge). Purges `*_frames`/staged/proxy/handoff/tmp; keeps CSV/DB/reels. Companion to the runner self-clean (2.2). See `[[cache-hygiene-policy]]` memory.
194   - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure = "unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires `reencode=True`.
195   - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.
196
197   ### 2.5 Validators ?
198   - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ? registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path, config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a validator**: subclass + `registry.register(MyValidator())`.
199   - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator, ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator, RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global registry as side effect).
200   - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring `'mp4'` (so MOV-family passes); external `ffprobe`.
201   - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator` (`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if absent). `min_fps` 29.9; min res 1280x720.
202   - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator` (`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded (comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES (too short).
203   - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail threshold.
204   - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var < `min_blur_threshold`(default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ? magic absolute, not resolution-normalized.
205   - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`; 3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport -> empty cfg, falls back to defaults, does NOT fail.
206
207   ### 2.6 Sports / Providers / Annotations ?
208   - `@backend/sports/base.py::SportProfile` ABC (`name`,`get_prompt`,`get_relevance_config`); `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg); `@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ? `register` instantiates profile to read `.name`). ? **add a sport**: subclass `SportProfile`, `sport_registry.register(MyProfile)`. § prompt emits JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS; `get()` returns a fresh instance each call.

209   - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`); `@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`; upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers `'gemini'`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key, model_name)`. § `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list, metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)` on ANY failure (empty != error); oversize clip refused LOUDLY (`logger.error` + `metrics['oversize_skipped']=True` ? callers aggregate into the report's `oversize_clip_skipped` rule); orphaned remote file possible if process dies mid-run.
210   - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`, `::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off `provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` -> LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`), `annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch -> List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable.

5. assistant (2026-06-19T15:46:40.861Z)

This is a large file. Let me read the rest of CODE_MAP.md and start verifying the refactor claims in parallel.

6. user (2026-06-19T15:46:42.732Z)

211   - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; -> `backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
212   - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub; injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only because no-arg ctor succeeds.
213   - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of file_provider/automated_provider are load-bearing (removing de-registers providers).
214
215   ### 2.7 Infrastructure (DB) ?
216   - `@backend/infrastructure/database.py::Database` ? SQLite, 6 tables (`videos/play_segments/events/candidate_segments/jobs/user_models`), `PRAGMA user_version` migration ladder in `::_init_db` (currently =**5**: `version<1` creates `candidate_segments`; `version<2` creates `jobs` + `idx_jobs_status` ? PR-1/#16; `version<3` ALTERs `jobs` to add `policy`/`next_poll_at` + `idx_jobs_due` (self-scheduling EXECUTION_POLICY); `version<4` ALTERs `videos`+`candidate_segments` to add a NULLABLE `user_id` (NULL=local install) + partial `idx_videos_user`/`idx_candidates_user`; `version<5` creates `user_models` + `idx_user_models_user_version`/`idx_user_models_one_active` (per-user model store, PERSONALIZATION_PLAN ? persisted/loaded but NOT yet read on the served path)). `::_get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_segment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`,`clear_video_data`,`segment_watermarks`,`prune_segments`; job store: `create_job`,`mark_job_running`,`set_job_progress`,`set_job_video_id`,`mark_job_done(result)`,`mark_job_failed(error)`,`get_job` (deserializes `result`/`player_pool` JSON),`fail_stale_jobs` (startup sweep -> count). ? candidate_segments is the detector-agnostic fine-tuning extension point (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index `idx_candidates_video_detector`). ? `add_video` is an **UPSERT** (`INSERT ... ON CONFLICT(id) DO UPDATE`) that mutates the row in place ? it does NOT cascade-delete dependent segments on re-ingest (that was the old `INSERT OR REPLACE` footgun); clearing segments is now an explicit destroy-AFTER-confirm via `segment_watermarks`/`prune_segments` (or `clear_video_data`). Events column is `type` not `event_type`; `query_play_segments` filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.
217
218   ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
219   - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed

config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if unavailable/disabled; pulls play_segments/candidates from `db` (read-only `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s `finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video. `::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default True.
220   - `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`. `::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now also `audio_present` for Input Quality reports extension; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present,oversize_clips_skipped}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). `record_run(run_signals=)` carries `run_signals.pop(video_id)` counters into `_ai_stage`. ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners). `::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so pure helpers like video_meta are importable for tests/coverage even without the dep.)
221   - `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules (9): `silent_provider_noop`(error: no api key + 0 segs), `oversize_clip_skipped`(error: provider refused chunks over MAX_UPLOAD_MB ? 4K stream-copy footgun), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `audio_disabled_or_missing`(warn: no audio stream -> loses the shuttle-thwack recall cue; reads `video_meta.audio_present`), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).
222   - `@backend/reporting/run_signals.py` ? ? in-process, thread-safe channel pipeline-internals -> report (`::incr(video_id,key,by)`, `::pop(video_id)`). No obsreport dep (always importable). Segmenter worker threads record facts the harvest can't see (e.g. `oversize_clips_skipped`); `emit_indexer_report` pops UNCONDITIONALLY (even reporting-off, so nothing accumulates) and passes to `record_run`. ? process-local best-effort: signals are dropped (by design) if no report is emitted for that video_id.
223
224   ---
225
226   ## 3. DATA CONTRACTS (canonical shapes ? one place)
227
228   § **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
229   `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
230   -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
231
232   § **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo `_extract_action_windows_from_chunk`):
233   `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float, mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
234
235   § **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
236   `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)), stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`

(stats/detection_params deserialized on read).
237   `detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
238
239   § **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
240   `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
241
242   § **jobs row** (`db.create_job` / `get_job` ? #16, PR-1): `{id:TEXT(uuid4.hex),
video_path, video_id?:TEXT(set once hashed), segmenter?, player_pool?:JSON list, max_frames?,
status:'queued'|'running'|'done'|'failed' (EXECUTION state ? pipeline verdict is
result["status"]), progress?:TEXT(stage label), error?:TEXT, result?:JSON(the former sync
/api/process response incl. reports), created_at, started_at?, finished_at?}`.
243
244   § **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
245
246   § **GT / golden CSVs** (TWO conventions ? do not conflate):
247   - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
248   - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
249
250   § **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
251   § **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
252   § **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},
...]}`; ? DB key = file MD5, manifest `video_id` is a label.
253
254   § **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
255
256   § **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
257
258   § **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
259
260   § **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
261
262   ---
263
264   ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
265   - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is

single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
266   - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)
267   - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
268   - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)
269   - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path, center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
270   - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
271
272   ---
273
274   ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified here)
275   - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
276   - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@scripts/run_phase3_eval.py`.
277   - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` / `@backend/tools/rally_slicer.py`.
278
279   ---
280
281   ## 6. GOTCHAS (cross-cutting footguns)
282   - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single `@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all run_*.py. Manifest `video_id` is a label, NOT the DB key.
283   - **Config access:** `backend/main.py` and all four `run_*.py` load via `backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in `main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
284   - **Segmenter default:** single source of truth = model `IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships `gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'` getattr fallback noted above.
285   - **`AppConfig.get` must return dict sections, not models:** the legacy `.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also silently keeps typo'd config keys (no rejection).
286   - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI `--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same `global_config.output.output_dir` ? honored end-to-end.
287   - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in the report.
288   - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-op.
289   - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.

- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
- **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
- **`add_video` is an UPSERT** (`INSERT ... ON CONFLICT(id) DO UPDATE`) that mutates the row in place ? re-ingesting a `video_id` does NOT cascade-delete its segments (the old `INSERT OR REPLACE` + FK-cascade wipe is gone). Segment clearing is now an explicit destroy-AFTER-confirm via `segment_watermarks`/`prune_segments` (or `clear_video_data`). `_get_connection` MUST still re-issue `PRAGMA foreign_keys=ON` per connection (CASCADE/SET NULL still apply on an explicit delete).
- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.
- **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
- **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`; `harness.get_predictions` reused by `regression.regress`.
- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
- **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.

---

7. RAMP PATHS (to do X -> open Y::symbol)

Task	Open
Change a config default / add a config field	`@backend/config/models.py` (single source of truth) -> regenerate via `scripts/doctor.py`/`save_default_config`; ? config.json may override (default_segmenter)
Read config in new code	`from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load`
Tune candidate windowing (velocity/merge/min-dur)	`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration`
Add a segmenter	subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/pipeline/segmenters/__init__.py`; if AI-based add to `@backend/reporting/harvest.py::_AI_SEGMENTERS`
Change stitching padding	`config.json stitching.{begin_padding,end_padding}`; impl `@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; wiring `@backend/main.py` (`/api/compile` + CLI trim); mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING`
Reclaim disk / clean caches	`python -m tools.cleanup_caches` (dry-run) then `--apply`; runs also self-clean via `indexing.{wasb,tracknet}.keep_frames=false` (default) +

`min_free_gb` guard; tool `@tools/cleanup_caches.py` |
316 | Filter highlights by shot count | `config.json stitching.min_shot_count` ? enforced on
BOTH `@backend/main.py::compile_highlights` (`/api/compile`) and
`@scripts/run_process_and_stitch.py` (segments without `shot_count` metadata are exempt) |
317 | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
318 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', Cls)` |
319 | Add an annotation/GT tier | subclass
`@backend/annotations/base.py::AnnotationProvider`; register + import in
`@backend/annotations/__init__.py` |
320 | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules`
(declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in
`@backend/reporting/harvest.py` |
321 | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage
builders; emit point is `@backend/main.py` `finally:` ->
`@backend/reporting/__init__.py::emit_indexer_report` |
322 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main`
(`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
323 | Distill Gemini -> local (thresholds / learned) |
`@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main`
(learned classifier); both `--manifest` |
324 | Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main`
(`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
325 | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
326 | Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) |
`backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has
play_segments (CLI, notebook, future API) |
327 | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@scripts/run_process_and_stitch.py::main` (edit hardcoded paths) |
328 | Score DB preds vs GT (one video) | `@run_eval.py::main` |
329 | Batch score a collector manifest | `@scripts/run_phase3_eval.py::main` (`--manifest`)
|
330 | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
331 | Guard rally-quality nightly (no GPU) | `@scripts/nightly_regression.py::main` ? re-
score golden corpus (default + milestone configs) vs `eval_baselines/nightly_baseline.json`;
exit 1=regress, 4=stale; `--update-baseline` to re-snapshot after an intended change. Scheduled
via `scripts/register_nightly_task.ps1` |
332 | Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/`
directly |
333 | Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
334 | Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;
producers before consumers) |
335 | Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db`
migration ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
336 | Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
337 | Probe an audio signal (diagnostic) | `@backend/eval/audio/e1_probe.py::main`; detector
in `@backend/pipeline/audio/hit_detector.py` |
338
339   ---
340
341   ## 8. TEST MAP (`tests/test_*.py` -> subsystem)
342   All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked.
`@run_all_tests.py` (repo root) is the suite runner.
343 | Test | Covers |
344 |---|---|
345 | `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
346 | `test_yolo_hybrid.py` | `pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`

ORCHESTRATION (mocks the `self.person` cue seams; candidate link; `store_yolo_candidates=False` skip; output-unchanged = the fusion-P1 regression guard) |
347 | `test_person_detector.py` | `pipeline/detectors/person_detector.PersonDetector` (`_PERSON_CLASS_ID` filter + ByteTrack handoff; `extract_action_windows_from_chunk` enriched dicts; `detections_per_frame` per-frame map + height) |
348 | `test_fusion_features.py` | `pipeline/detectors/fusion_features` (pure feature math: point-in-polygon, in-court counts, players/ratio, motion, two-sided w/ polygon, colocation halo + alignment) |
349 | `test_fusion_hybrid.py` | `pipeline/segmenters/fusion_hybrid.FusionSegmenter` (net_crossings persisted; superset; Gate A & Gate B handoff gating; person-cue features; `_build_runner` substrate select) |
350 | `test_tracknet_hybrid.py` | `pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter` (4-phase; healthcheck-abort; `_probe_fps_width` fallback) |
351 | `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked path, per-chunk offset, `shot_count`<-`shuttle_exchange_count`, chunk re-encode/`ai_chunk_scale_width` args, oversize-skip -> run_signals + ERROR log) |
352 | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`, `from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck, run_predict, stage) |
353 | `test_wasb_runner.py` | `pipeline/detectors/wasb_runner.WasbRunner` WSL-glue (DetectorRunner conformance, healthcheck, `parse_trajectory_csv`/`trajectory_to_action_windows` rebind, run_predict; WSL mocked) |
354 | `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing count, gate kills single-toss) |
355 | `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize, contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
356 | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
357 | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate, `write_scaffold` |
358 | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict; also `@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
359 | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate (micro/macro, zero-div) |
360 | `test_calibration.py` | `eval.calibration` pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
361 | `test_calibrate_wasb.py` | `eval.calibrate_wasb` load_golden_gts/make_predict_fn/grid/run, BASELINE |
362 | `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-one-video-out |
363 | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV schemas, mmss; export->slicer reuse |
364 | `test_eval_training.py` | `eval.training` label_candidates / feature extraction / build_training_set |
365 | `test_eval_classifier.py` | `eval.classifier.LogisticRegression` fit/predict_proba/balanced/JSON round-trip |
366 | `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also `@run_regression.py::main` (exit-code CI gate) |
367 | `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies / learned-vs-baseline |
368 | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure helpers, no API) |
369 | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope, _moving_stats) + `eval/audio/e1_probe.cluster_hits` |
370 | `test_compiler.py` | `pipeline/stitcher/compiler.VideoCompiler` (`_parse_time` formats, re-encode->concat path, boundary validation; ffmpeg/ffprobe mocked) |
371 | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label parse |
372 | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
373 | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU |
374 | `test_database.py` | `infrastructure.database.Database` schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
375 | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig defaults pinned, `extra` tolerance, `.get()` dict-compat) |

```
376     | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
`harvest.record_run`); ? `importorskip("obsreport")`; all 9 footgun rules + plain-dict-OR-
AppConfig + `report.enabled=False` |
377     | `test_run_signals.py` | `reporting/run_signals` (incr/pop consume + per-video
isolation; emit consumes signals even with reporting OFF). ? deliberately NO obsreport dep ?
this is the half of the oversize channel CI can verify (obsreport is absent there) |
378     | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
integrity (every `faq_ref` resolves; actionable rules cite a faq_ref); ?
`importorskip("obsreport")` |
379     | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
380     | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
381     | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard refuses LOUDLY w/ `oversize_skipped` metric,
non-list JSON -> [], cleanup delete) |
382     | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
383     | (no dedicated test) | individual `pipeline/validators/*` impls, `main.py`,
`scripts/run_process_and_stitch.py`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
384
```

7. user (2026-06-19T15:46:42.746Z)

```
0       - [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md) ? Latest (v8) vendor
labeling guidelines mirrored from the collector's `docs/annotation/`, with the fusion-driven
court-calibration / singles-doubles additions marked `? ADDED (fusion)`.
```

8. assistant (2026-06-19T15:46:47.529Z)

```
Now let me read the full README.md from the beginning to see the doc-index and module-structure
claims.
```

9. user (2026-06-19T15:46:48.831Z)

```
29 docs/README.md
```

10. assistant (2026-06-19T15:46:51.341Z)

```
The README I read started at line 0, but it has 29 lines total and the content I saw starts mid-
doc. Let me re-read it properly from line 1.
```

11. user (2026-06-19T15:46:52.383Z)

```
1       - [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md) ? Latest (v8) vendor
labeling guidelines mirrored from the collector's `docs/annotation/`, with the fusion-driven
court-calibration / singles-doubles additions marked `? ADDED (fusion)`.
2       - [RALLY_QUALITY_RESEARCH.md](RALLY_QUALITY_RESEARCH.md) ? **? Foundation doc** for
closing the local-vs-Gemini rally-detection gap: the measured gap + the code-grounded over-merge
diagnosis ("shuttle moving ? rally"), an inventory of the eval/experiment harness + its gaps,
the strengthened evaluation framework (segmental Edit/F1@k that make over-segmentation first-
class, golden-corpus + active learning, Gemini-agreement as a *shadow* signal), an
**adversarially-verified, cited external-research synthesis** (TAL/TAS boundary heads, racket-
sports rally-state cues, small-data, eval rigor), a **prioritized cheap?durable roadmap with
falsifiable success criteria**, and (§7) a **tracked project plan** ? work-item checklist,
sequencing, the single quality **number** attributed to the effort, and a **definition of done**
(project completes when all tiers land + the number is recorded). The plan that the
[QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md) log executes against; ties together
EXPERIMENT_HARNESS / MULTI_SIGNAL_FUSION_PLAN / ANALYZERS_AND_RECONCILERS /
OWNED_MODEL_IMPLEMENTATION_PLAN.
3       - [RALLY_DETECTION_GAP_CLOSURE.md](RALLY_DETECTION_GAP_CLOSURE.md) ? **Tracked execution
```

doc (2026-06-18)** for closing the remaining rally-detection accuracy gap **per video, before launch**, driven by the growing golden corpus. First ground-truth clip (GX010141) reframed it: boundaries are tight (R2/R3/R4 holds) ? the gap is **detection**: (G1) recall (local & Gemini both miss ~half on hard clips, and *different* rallies ? complementary), (G2) precision (local over-fires), (G3) **rally-END over-extension on the quick-return-after-point** (shuttle still moving ? needs a *player-state* signal). Levers (owner-gated): player-kinematics + shuttle-in-hand / sustained-invisibility end-rule with reverse-timeline backtrack, `rally_gate` Gate A wiring, complementary fusion. **Peer** of `RALLY_QUALITY_RESEARCH.md` (execution tracker, not research); §7 is the source of truth.
- RALLY_DETECTION_QUALITY_REPORT.md ? **Technical-paper checkpoint (2026-06-17)** of rally-detection quality: the honest measured state (over-segmentation milestone tolerance-F1 0.091?0.323 at n=11, p=0.002; merge-rate 0.694?0.150; first real-footage ground-truth at-par with START at Gemini parity), the task-faithful R2 metric, a detailed Gemini comparison, next steps + expected wins, areas yet to be explored, and a publishability / path-to-submission verdict. Renders to a polished PDF; the externally-shareable "toward a paper" summary of `RALLY_QUALITY_RESEARCH.md` + `QUALITY_ITERATIONS.md`.
- EVALUATION.md ? How we measure and improve quality.
- EXPERIMENT_HARNESS.md ? Design (P1 SHIPPED) for A/B + shadow + SxS experimentation on rally-detection variants with **zero production regression** ? experiments are decoupled from deploy (new cues default OFF, promote only on measured LOVO lift, rollback = flip a flag). Composes the existing `backend/eval/` machinery (`calibration`, `regression`, `metrics`); implemented in `backend/eval/experiment.py`.
- QUALITY_ITERATIONS.md ? **Living log** of rally-detection quality iterations (hypothesis ? what shipped ? measured/expected effect ? lesson). The reverse-chronological record; terminal-state snapshots get archived under `archives/quality-iterations/`.
- ANALYZERS_AND_RECONCILERS.md ? Design (owner-gated, NOT started): a **signal-fusion framework** for rally detection ? producers emit confidence-scored **signals** at three temporal **scopes** (frame detectors / window analyzers / session analyzers), a learned **scorer** weights them, and an auditable **report-first reconciliation** pass lints the decision against the evidence. The spine: **no special-casing in the decision path** (signals?scorer, never `if/else`). Worked examples: a service-fault analyzer, a warm-up/practice span detector, and consuming a per-frame shuttle-state detector. A peer to `EXPERIMENT_HARNESS.md` and `MULTI_SIGNAL_FUSION_PLAN.md` (ADR-007 modular rally layer).
- PERSONALIZATION_PLAN.md ? Design (owner-adopted **hybrid**; Phase-0 landing) for the **per-user personalization feature on a generalization-first baseline**: a thin per-user *tuning* layer gated by the honest small-N statistics (a **min-N promotion floor** + a **structural-guard exemption**, never a service gate) + a **contribution flywheel** (free-tier videos pool into the owner-trained baseline; *"a tool that gets better the more you help it"*). Records the **locked owner decisions** (identity, storage, `min_n` semantics, Gate-A default, free-tier pooling). Built so far: per-user promotion gate (#141) + held-out-USER learning curve (#142).

- GOLDEN_REGRESSION_FIXTURES.md ? **Design (owner-gated, NOT started):** video-free, non-attributable regression fixtures so a clean clone runs the rally-seg suite with **no video/GPU/DB**. Two layers ? the post-detector freeze + **dual gates** (characterization + quality-floor) reusing the `backend/eval/` stack, and the **SEALED-FIXTURES** privacy filter (de-attribution-at-source, tiered public/key-gated, shuttle-only public tier). Carries a cited **IN/US/EU legal memo** (derived-data IP/privacy), a threat model, and a **§7 progress tracker + definition-of-done**. Extends GOLDEN_DATA_SHARING.md.

Product / platform plans

- DESKTOP_APP_PLAN.md ? **Tracked scoping doc (2026-06-18, owner-gated, `[design]` ? NOTHING built):** package the local pipeline as a **cross-platform desktop app** for non-technical players ? plug in the camera's USB/SD card, click once, get back **only the highlight reels**, fully on-device (no upload, no original-video bloat). The centerpiece is the **de-WSL native-compute port** (run WASB natively per-OS: Linux/Windows CUDA · macOS MPS/CPU ? drop WSL2), and its **make-or-break unknown**: is CPU acceptable on a typical *no-GPU* enthusiast laptop (WASB is ~3.4× real-time on a laptop 2070S)? §7 tracker = **P0 compute-feasibility spike ? P1 native detector ? P2 app shell ? P3 packaging/installers**. Realizes the **L0 fully-local** tier of VIDEO_LOCALITY_MODEL.md; is the **`LocalDesktop` ComputeBackend** of PLATFORM_ARCHITECTURE.md §3.2; the smaller/export-clean owned model (OWNED_MODEL_IMPLEMENTATION_PLAN.md) is one answer to the

```
compute crux.
15
16      ## Practical / ongoing
17      - [TRACKNET_WSL_SETUP.md](TRACKNET_WSL_SETUP.md)
18      - Golden set docs (`GOLDEN_SET_*`)
19      - Evaluation harness and calibration drivers (in `backend/eval/`)
20      - [GOLDEN_DATA_SHARING.md](GOLDEN_DATA_SHARING.md) ? How the GPU box and CPU dev/agent
sessions share golden artifacts (trajectory CSVs, `*_golden_features.json`) via a OneDrive
folder; the GPU-extract ? CPU-litmus split. See **#175**.
21
22      ## Conventions & templates
23      - [PROJECT_DOC_TEMPLATE.md](PROJECT_DOC_TEMPLATE.md) ? Reusable skeleton for a **tracked
project doc**: status header · decisions-locked · progress tracker (?/?/?) · definition-of-done
· archived on completion. Standardizes the emergent pattern in `RALLY_QUALITY_RESEARCH.md` §7
and `GOLDEN_REGRESSION_FIXTURES.md`. Use for any substantial design/project work.
24
25      ## Historical documents
26      Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).
27
28      - [CODE_AUDIT_AND_TEST_HARDENING.md](CODE_AUDIT_AND_TEST_HARDENING.md) ? **COMPLETED**
(2026-06-14; T1?T5 + full loop + verification per #162). Living record of the June 2026 plumbing
/ contract / architectural audit, 626-test infrastructure, positive test plans for the 5
identified issues, multi-layer regression protocol (including ablation/experiment layer), and
pre/post-change verification checklist. See `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-
COMPLETED-2026-06-14.md` and `docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md`
for the final archived state. (Top-level is now pointer stub per review; full content in
archives/.)
29
30      See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md`
for classification of small tech debt / starter projects.
```

12. assistant (2026-06-19T15:46:58.599Z)

```
The README.md doc-index doesn't reference CODE_MAP.md by name, but the CLAUDE.md says README is
the doc index. Now let me verify all the refactor claims. Let me check the key refactored files
mentioned in my slice (#226-#235, #215, #220) and confirm CODE_MAP's claims about module
structure.
```

13. user (2026-06-19T15:46:59.593Z)

```
backend\eval\ablation_pdf.py
```

14. user (2026-06-19T15:46:59.758Z)

```
backend\api\__init__.py
backend\api\uploads.py
backend\api\job_worker.py
```

15. user (2026-06-19T15:47:00.219Z)

```
backend\eval\golden_real_fixtures.py
```

16. user (2026-06-19T15:47:00.704Z)

```
backend\eval\golden_fixtures.py
```

17. user (2026-06-19T15:47:01.145Z)

```
backend\eval\rally_seg_eval.py
```

18. user (2026-06-19T15:47:01.432Z)

```
tests\test_golden_real_fixtures.py
```

19. assistant (2026-06-19T15:47:05.573Z)

Several refactored modules exist (`backend/api/uploads.py`, `backend/api/job_worker.py`, `backend/eval/ablation_pdf.py`, `backend/eval/golden_real_fixtures.py`). Now let me check which are referenced in CODE_MAP and which are NOT. The CODE_MAP §2.3 mentions `golden_fixtures.py` and `tests/test_golden_real_fixtures.py` but I need to check whether it mentions `golden_real_fixtures.py` as a module. Let me grep CODE_MAP for these.

20. user (2026-06-19T15:47:07.135Z)

115:[Omitted long matching line]
183:[Omitted long matching line]

21. user (2026-06-19T15:47:08.045Z)

472:def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
483:        raise ValueError("refresh_snapshot needs --slug S or --all")
777:    if args.refresh_snapshot:
785:        targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
790:    if args.freeze_real:
792:        # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always has.
801:            expected = golden_real_fixtures.freeze_real_fixture(
808:        except golden_real_fixtures.RefusalError as e:
834:    "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",

22. assistant (2026-06-19T15:47:11.967Z)

Found a key finding: #233 (`golden_real_fixtures.py`) extracted `freeze_real_fixture`/`RefusalError`/`refresh_snapshot` out of `golden_fixtures.py` into a separate module, but CODE_MAP §2.3 line 183 still attributes `freeze_real_fixture` and `refresh_snapshot` to `golden_fixtures.py`. Let me look at line 183 closely and verify what's actually in each file.

23. user (2026-06-19T15:47:13.408Z)

```
460         for spec in specs:
461             fdir = fixtures_dir / spec.slug
462             fdir.mkdir(parents=True, exist_ok=True)
463             rows = make_synthetic_trajectory(spec)
464             write_trajectory_csv(rows, fdir / "trajectory.csv")
465             write_labels_csv(spec.gts, fdir / "labels.csv")
466             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
467             slugs.append(spec.slug)
468         _write_manifest(specs, fixtures_dir)
469         return slugs
470
471
472     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
473                          fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
474         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
475         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
476         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
477         if all_slugs:
478             targets = [d.name for d in sorted(fixtures_dir.iterdir())
479                        if d.is_dir() and (d / "trajectory.csv").is_file()]
480         elif slug:
481             targets = [slug]
482         else:
483             raise ValueError("refresh_snapshot needs --slug S or --all")
484         for s in targets:
```

```
485             freeze_fixture(s, fixtures_dir=fixtures_dir)
486         return targets
487
488
489     # ============================================================================================
490     # Manifest entry derivation (shared ? used by both the synthetic and real-fixture
writers)
491     # ============================================================================================
492
493
494     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
495         """A manifest row for a single fixture, derived from its provenance block."""
496         slug = prov["slug"]
497         return {
498             "slug": slug,
499             "fps": _round(float(prov["fps"])),
500             "frame_width": _round(float(prov["frame_width"])),
501             "windowing": dict(prov.get("windowing", {})),
502             "kind": prov.get("kind", "synthetic"),
503             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
504             "paths": {
505                 "dir": slug,
506                 "trajectory": f"{slug}/trajectory.csv",
507                 "labels": f"{slug}/labels.csv",
508                 "expected": f"{slug}/expected.json",
509                 "provenance": f"{slug}/provenance.json",
510             },
511         }
512
513
514     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
515         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
516         entries = load_manifest(fixtures_dir)
517         entries = [e for e in entries if e.get("slug") != prov["slug"]]
518         entries.append(_manifest_entry_for(prov))
519         entries.sort(key=lambda e: str(e.get("slug")))
```

24. user (2026-06-19T15:47:14.081Z)

```
1       """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3       This module holds the **real-fixture subsystem** split out of ``golden_fixtures`` (which
keeps the
4       shared synthetic/replay primitives). It turns an OWNED, minors-free real trajectory +
golden labels
5       into a committable, de-attributed Tier-0 fixture, reusing M1's ``replay_fixture`` for
the snapshot.
6       (docs/GOLDEN_REGRESSION_FIXTURES.md §4c anti-attribution, §5 threat model, §7 row P1, §6
honest
7       limits.)
8
9       The shared primitives (constants, float/JSON helpers, ``replay_fixture``, the CSV
writers, the
10      manifest appender) live in ``golden_fixtures`` and are imported FROM it here ? there is
a single
11      source of truth, no duplicated logic/thresholds. ``golden_fixtures`` re-exports the
public names
12      defined here (``freeze_real_fixture``, ``RefusalError``, ``_scan_forbidden``) at the
bottom of that
13      module so every historical ``golden_fixtures.<name>`` import path stays valid.
14
15      §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
```

```
16      are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
17      """
18      from __future__ import annotations
19
20      import os
21      import shutil
22      import tempfile
23      from pathlib import Path
24      from typing import Any, Dict, List, Optional, Sequence, Tuple
25
26      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-------------
27      from backend.eval.gt_loader import load_gt_intervals
28      from backend.eval.regression import gt_hash
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30
31      # --- shared primitives, imported FROM golden_fixtures (single source of truth)
---------
32      from backend.eval.golden_fixtures import (
33          CURRENT_SCHEMA,
34          FIXTURES_DIR,
35          FORBIDDEN_KEYS,
36          PINNED_MAX_WINDOW,
37          PINNED_MERGE_GAP,
38          PINNED_MIN_WINDOW,
39          PINNED_VT,
40          Interval,
```

25. assistant (2026-06-19T15:47:19.290Z)

Now I understand the #233 split. `golden_fixtures.py` still has `refresh_snapshot` and re-exports `freeze_real_fixture`/`RefusalError` from `golden_real_fixtures.py`. So the CODE_MAP line 183 claim that `freeze_real_fixture` is `@backend/eval/golden_fixtures.py` is technically stale (it's defined in `golden_real_fixtures.py`, re-exported). Let me read CODE_MAP line 183 in full and verify the exact wording.

26. user (2026-06-19T15:47:20.467Z)

```
183     - `@backend/eval/golden_fixtures.py` ? **video-free regression harness**
(`docs/GOLDEN_REGRESSION_FIXTURES.md`). `::replay_fixture` =
`tracknet_runner.parse_trajectory_csv` ? `distill_local.build_candidates`(PINNED preset) ?
`experiment.VideoCandidates`(5 shuttle cols only) ? `experiment.predict(CONTROL)` ?
`metrics.score`; **CONTROL / pure-stdlib ? no numpy on the replay path** (cross-OS stable; the
learned/numpy path is deliberately excluded). `::freeze_synthetic` (3 committed synthetic Tier-0
fixtures), `::freeze_real_fixture` (P1/M2: `RefusalError` unless `owned`+`minors_free`+non-blank
`license`; opaque random `fx_<hex>` slug, metric-invariant time-origin reset, STRICT
`gt_loader`, positive no-`FORBIDDEN_KEYS`/MD5/source-path scan), `::refresh_snapshot`,
`::compare_expected` (exact-int / approx-1e-6, NEVER byte-compare),
`::load_manifest`/`::load_fixture` (schema_version dispatch). CLI `--freeze-
synthetic`/`--freeze-real`/`--refresh-snapshot`/`--check`. Tests:
`tests/test_golden_fixtures.py` (M1/M3 characterization) + `tests/test_golden_real_fixtures.py`
(P1/M2). CI: the `golden-crossos` job (ubuntu/windows/macos) runs `--check` + the M1 test. ?
synthetic-tier numbers = plumbing correctness, NOT field quality.
```

27. assistant (2026-06-19T15:47:30.095Z)

This is a borderline case. CODE_MAP §2.3 line 183 lists `::freeze_real_fixture` as a symbol of `@backend/eval/golden_fixtures.py`. Per #233, `freeze_real_fixture` now lives in `golden_real_fixtures.py` and is only re-exported from `golden_fixtures.py`. Since the `::sym` notation per the LEGEND means "symbol in the nearest preceding `@path`", and `golden_real_fixtures.py` is a NEW module entirely absent from the map, this is stale. The map "purports to be complete" per the instructions. Let me note this and the missing-module concern.

Now let me verify the other refactor claims. Let me check #226 (ablation_pdf), #234 (job_worker), #228 (uploads ? already confirmed in CODE_MAP), #230 (motion halves), #232 (db _execute_write), #235 (shared golden-feature loader), #215 (_is_fusion_schema in experiment.py).

28. user (2026-06-19T15:47:31.164Z)

No matches found

29. user (2026-06-19T15:47:32.407Z)

```
114:[Omitted long matching line]
216:[Omitted long matching line]
242:§ **jobs row** (`db.create_job` / `get_job` ? #16, PR-1): `{id:TEXT(uuid4.hex), video_path,
video_id?:TEXT(set once hashed), segmenter?, player_pool?:JSON list, max_frames?,
status:'queued'|'running'|'done'|'failed' (EXECUTION state ? pipeline verdict is
result["status"]), progress?:TEXT(stage label), error?:TEXT, result?:JSON(the former sync
/api/process response incl. reports), created_at, started_at?, finished_at?}`.
```

30. assistant (2026-06-19T15:47:37.181Z)

Now let me check the #234 job_worker refactor. CODE_MAP §2.0 line 114 describes `_run_job`, `_execute_processing`, `_get_executor` as living in `main.py` (`@backend/main.py`). Let me verify whether these moved to `backend/api/job_worker.py`.

31. user (2026-06-19T15:47:38.708Z)

```
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
33:def _get_executor() -> ThreadPoolExecutor:
46:    is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
86:        result = _main._execute_processing(
```

32. user (2026-06-19T15:47:39.764Z)

```
65:# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
66:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
71:from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
75:    _get_executor,
125:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
267:    _get_executor().submit(_drain_due_jobs)
637:        _get_executor().submit(_drain_due_jobs)
```

33. assistant (2026-06-19T15:47:45.318Z)

So #234 moved `_run_job`, `_get_executor`, `_drain_due_jobs`, etc. to `backend/api/job_worker.py`, but `_execute_processing` stays in `main.py` (line 125). CODE_MAP §2.0 line 114 still says `::_run_job` and `::_get_executor` are in `@backend/main.py`. They are re-exported there (line 71-75), so `backend.main._run_job` still resolves. This is a moved-symbol situation where the canonical definition moved but back-compat re-export keeps the path valid. Let me read the job_worker.py file fully to confirm exactly which symbols moved, and read main.py lines 60-130 to confirm re-exports.

34. user (2026-06-19T15:47:45.945Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
```

```
    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
    `backend.main` module object at call-time (`import backend.main as _main`), never as a
    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
    ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
    inside the functions also avoids the circular import at module load (main.py imports
this
    module; this module must not import main at import time).
    """
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
```

```
        )


    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main
        db_instance = _main.db_instance
        job_id = job["id"]
        req = _main._job_to_request(job)
        try:
            result = _main._execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
            if result.get("video_id"):
                db_instance.set_job_video_id(job_id, result["video_id"])
            db_instance.mark_job_done(job_id, result)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _drain_due_jobs() -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main
        ran = 0
        while True:
            job = _main.db_instance.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting()
        return ran


    def _schedule_next_drain_if_waiting() -> None:
```

```
125         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126         self-scheduled job resumes with no further client submit. The drain itself re-checks
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return  # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153
```

35. assistant (2026-06-19T15:47:51.132Z)

Now this is a significant finding. The job worker has been substantially reworked (#234 and an EXECUTION_POLICY P3 WAIT_AND_RESUME change). Note: the actual worker function is now `_run_claimed_job` (claiming via `claim_due_job` / `_drain_due_jobs`), NOT `_run_job` as CODE_MAP line 114 describes. Let me verify whether `_run_job` still exists anywhere, and check the `/api/process` flow described in line 114 (which says `_get_executor().submit(_run_job,...)`).

36. user (2026-06-19T15:47:52.692Z)

```
66:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
74:    _drain_due_jobs,
241:def process_video(req: ProcessRequest):
247:    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
264:    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
267:    _get_executor().submit(_drain_due_jobs)
637:        _get_executor().submit(_drain_due_jobs)
```

37. user (2026-06-19T15:47:53.079Z)

```
Found 3 files
docs\CODE_MAP.md
tests\test_job_worker_loop.py
docs\EXECUTION_POLICY.md
```

38. assistant (2026-06-19T15:47:57.697Z)

Confirmed: `_run_job` no longer exists in the code (only appears in CODE_MAP.md, EXECUTION_POLICY.md, and a test name). The actual flow is now `process_video` ? `db.create_job` ? `_get_executor().submit(_drain_due_jobs)` (line 264-267), and the worker is `_drain_due_jobs` ? `claim_due_job` ? `_run_claimed_job`. Let me read main.py lines 241-270 to get the exact current flow, and confirm the `_run_job` reference in CODE_MAP line 114 is genuinely stale code.

39. user (2026-06-19T15:47:58.900Z)

```
241     def process_video(req: ProcessRequest):
242         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
243
244         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
245         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
246         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
247         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
248         """
249         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
250         try:
251             validate_input_path(req.video_path, allowed_root=allowed_root)
252         except ValueError as e:
253             raise HTTPException(status_code=400, detail=str(e)) from e
254         if not os.path.exists(req.video_path):
255             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
256         if req.segmenter and not segmenter_registry.get(req.segmenter):
257             available = list(segmenter_registry.list_available().keys())
258             raise HTTPException(
259                 status_code=400,
260                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
261             )
262
263         job_id = uuid.uuid4().hex
264         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
265                                       req.player_pool, req.max_frames):
266             raise HTTPException(status_code=500, detail="Failed to persist job record.")
267         _get_executor().submit(_drain_due_jobs)
268         return JSONResponse(
269             status_code=202,
270             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
```

40. assistant (2026-06-19T15:48:08.628Z)

Confirmed. CODE_MAP line 114 describes the OLD `_get_executor().submit(_run_job,...)` flow with a `::_run_job` worker. The current flow submits `_drain_due_jobs` (a drain loop that claims jobs via `claim_due_job`), and the per-job worker is `_run_claimed_job`. The `::_run_job` symbol is dead. Also note: the jobs status now includes a `'waiting'` state (EXECUTION_POLICY P3 WAIT_AND_RESUME), not just `queued|running|done|failed`. This is a substantial staleness in line 114.

However, I need to be careful about scope. My slice is "CODE_MAP + README doc-index/module-structure staleness." The `_run_job` ? `_drain_due_jobs`/`_run_claimed_job` is a module-structure/symbol staleness from #234. Let me confirm the database migration ladder version too (line 216 says =5; jobs row status). Let me check the DB version and jobs status values.

41. user (2026-06-19T15:48:10.045Z)

```
67:        ``user_version`` stays un-bumped ? and the next open would re-run the same
86:            # Versioned migrations live here. PRAGMA user_version is the schema
89:            version = cursor.execute("PRAGMA user_version").fetchone()[0]
105:            # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 6
178:        cursor.execute("PRAGMA user_version = 1")
```

```
205:        cursor.execute("PRAGMA user_version = 2")
209:        # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is due
210:        # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
211:        # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
215:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
216:        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
217:        cursor.execute("PRAGMA user_version = 3")
237:        cursor.execute("PRAGMA user_version = 4")
275:            CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
285:        cursor.execute("PRAGMA user_version = 5")
559:    def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
560:        """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now, releasing
the
561:        worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
564:            "UPDATE jobs SET status='waiting', "
565:            "next_poll_at=datetime('now', ?), "
572:        """Seconds until the soonest parked ('waiting') job becomes due, so the worker can
574:        already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578:                    "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579:                    "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS secs "
580:                    "FROM jobs WHERE status='waiting'").fetchone()
588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
589:        """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose ``next_poll_at``
is due ?
598:                    "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599:                    "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1").fetchone()
```

42. assistant (2026-06-19T15:48:17.711Z)

DB version is still 5, so line 216's `=5` is correct. But the jobs status now has a ``'waiting'`` state which CODE_MAP §2.0 line 114 and the jobs-row data contract (line 242) both omit (they list only `queued|running|done|failed`). The §2.7 line 216 already mentions `version<3` adds `policy`/`next_poll_at` self-scheduling EXECUTION_POLICY, so the DB section is roughly current. But line 114 and 242's status enums are now stale.

Let me focus on the highest-confidence, in-scope (CODE_MAP/README module-structure) findings. Let me now verify the other refactors: #226 ablation_pdf (is it referenced anywhere in CODE_MAP?), #230 motion halves, #232 db _execute_write, #235 shared golden-feature loader, #220 long-rally lens, #231 _build_preset, #215.

Let me check whether CODE_MAP claims about these modules contradict the new structure. First, does CODE_MAP reference an "ablation" export anywhere, or `rally_seg_eval`?

43. user (2026-06-19T15:48:19.776Z)

```
123:[Omitted long matching line]
299:- **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API**
for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV
schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`;
`harness.get_predictions` reused by `regression.regress`.
326:| Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) |
`backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has
play_segments (CLI, notebook, future API) |
```

44. user (2026-06-19T15:48:19.817Z)

```
No files found
```

45. user (2026-06-19T15:48:20.274Z)

backend\pipeline\segmenters\motion.py

46. assistant (2026-06-19T15:48:25.129Z)

Key observations:
- `ablation_pdf.py` (#226), `rally_seg_eval.py`, `golden_real_fixtures.py`, `job_worker.py`, `motion_detection.py` (a #230 split?) ? none referenced in CODE_MAP except via the nightly_regression reference to `rally_seg_eval.evaluate` (line 123). These are new modules absent from the §2 module list.

Let me check #230 (motion detection/populate halves) and whether motion.py was split, and check what modules exist in segmenters/detectors that CODE_MAP §2.1/2.2 claims completeness about.

47. user (2026-06-19T15:48:25.800Z)

backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters__init__.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py

48. user (2026-06-19T15:48:26.289Z)

backend\pipeline\detectors__init__.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\detectors\audio_features.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\tracknet_runner.py

49. user (2026-06-19T15:48:26.427Z)

backend\eval__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\gemini_refine.py
backend\eval\export_wasb_segments.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\eval\batch.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\eval\served_gate_a.py

```
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\eval\segmentation_metrics.py
backend\eval\tolerance_metrics.py
backend\eval\windowing.py
backend\eval\ablation_pdf.py
backend\eval\ablation.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\rally_seg_eval.py
```

50. assistant (2026-06-19T15:48:38.067Z)

This is important. There are MANY `backend/eval/` modules NOT in the CODE_MAP §2.3 list. The CODE_MAP §2.3 lists only a subset. But my slice is specifically about the recent refactors (#226-#235, #215, #220) and "new modules absent from the map if the map purports to be complete." The CODE_MAP §2.3 header says "Eval & calibration (all pure core; I/O in CLI drivers)" but doesn't explicitly claim exhaustive completeness. However, the regenerate note says "parallel subsystem readers ? this doc."

Let me be careful and focused. The instructions say flag new modules absent from the map only "if the map purports to be complete." The §2 SUBSYSTEMS sections list specific files. Many eval modules (`gt_loader`, `eval_stats`, `score_calibration`, `fusion_golden`, `per_user_*`, `served_gate_a`, `promotion`, `serve_contrast`, `splits`, `segmentation_metrics`, `tolerance_metrics`, `windowing`, `ablation`, `ablation_pdf`, `fusion_audio`, `fusion_compare`, `rally_seg_eval`, `golden_real_fixtures`) are absent. This is too many to be a refactor-staleness issue ? it reflects the doc being a curated map, not exhaustive. The map explicitly says "Regenerate: rerun the build-code-map workflow" ? it's not claiming line-by-line completeness of every file.

I'll restrict findings to genuine CONTRADICTIONS where CODE_MAP states a symbol/path that the code now contradicts due to a refactor. Let me verify the strongest candidates precisely. First, confirm `_run_job` is genuinely gone and the line-114 description contradicts code. Then check the `golden_real_fixtures` re-export nuance, line 240's play_segment metadata (`shot_count_confidence:'Unknown'`), and #220 wiring (`min_rally_duration` in StitchingConfig ? CODE_MAP line 130 lists StitchingConfig fields).

51. user (2026-06-19T15:48:39.499Z)

```
1       """Pydantic models for the application configuration.
```

52. user (2026-06-19T15:48:40.975Z)

```
239:class StitchingConfig(BaseModel):
240:    begin_padding: float = 0.5
241:    end_padding: float = 0.5
242:    use_cache: bool = False
243:    min_shot_count: int = 1
246:    # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
251:    min_rally_duration: float = Field(default=0.0, ge=0.0)
```

53. assistant (2026-06-19T15:48:47.134Z)

Confirmed: #220 added `min_rally_duration` to `StitchingConfig`. CODE_MAP line 130 lists `StitchingConfig` fields as `(begin_padding=0.5, end_padding=0.5, use_cache=False, min_shot_count=1)` ? omitting `min_rally_duration`. This is a stale/incomplete field list. However, omitting a newly-added field is "incomplete" rather than a hard "contradiction." The instructions say to flag wrong/missing paths and moved symbols, and new modules absent. A missing config field in an explicit field enumeration is borderline. Let me check whether

CODE_MAP claims that list is exhaustive ? it reads as an enumeration `::StitchingConfig (begin_padding=0.5,...,min_shot_count=1)`. Adding a field makes that enumeration incomplete but not contradictory. I'll consider it lower-priority.

Let me focus on the clearest contradiction: the `_run_job` flow. Let me read the exact CODE_MAP line 114 text to craft a surgical edit, and verify what the current names should be.

54. user (2026-06-19T15:48:47.595Z)

114 - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).

55. assistant (2026-06-19T15:48:55.896Z)

This line 114 contains multiple stale claims:
1. `_get_executor().submit(_run_job,...)` ? now `submit(_drain_due_jobs)`
2. `::_run_job (worker: mark running -> ...)` ? no longer exists; worker is `_drain_due_jobs` ? `claim_due_job` ? `_run_claimed_job`
3. job `status = EXECUTION state (queued|running|done|failed)` ? now includes `waiting` (WAIT_AND_RESUME / self-scheduling)

But wait ? I must check whether this is genuinely from the slice's listed refactors (#234) or a separate EXECUTION_POLICY change. The instructions list #234 `backend/api/job_worker.py`. The `_run_job` ? `_drain_due_jobs`/`_run_claimed_job` rename and the `waiting` state appear to predate or accompany #234 (EXECUTION_POLICY.md P3). Regardless, it's a genuine code-contradicting staleness in CODE_MAP's module/symbol description. Let me verify `fail_stale_jobs` still exists and the startup line, to keep my edit surgical and only change what's actually stale.

56. user (2026-06-19T15:48:56.965Z)

275:def get_job(job_id: str):
556:    swept = db_instance.fail_stale_jobs()

57. user (2026-06-19T15:48:57.896Z)

275:def get_job(job_id: str):
277:    (queued|running|done|failed); the pipeline verdict is result["status"]."""

58. assistant (2026-06-19T15:49:08.009Z)

Interesting ? even main.py's own `get_job` docstring (line 277) still says `(queued|running|done|failed)` omitting `waiting`. So the `waiting` status omission in CODE_MAP line 114 mirrors a docstring that also omits it. Since the code DB layer clearly supports `waiting` (`set_job_waiting`, `claim_due_job`), the CODE_MAP enumeration is incomplete but the in-code docstring agrees with it, making it a softer issue. I'll keep my focus on the unambiguous `_run_job` contradiction (the symbol genuinely doesn't exist).

The strongest, in-scope finding is the `_run_job` symbol description. Let me craft a surgical fix for the `_get_executor().submit(_run_job,...)` and the `::_run_job (worker...)` portions. The current accurate description: submit `_drain_due_jobs`; the worker drains via

`claim_due_job` and runs each job through `_run_claimed_job`.

Now let me verify the §2.7 line 216 DB section ? it lists job-store methods. Does it include `claim_due_job`/`set_job_waiting`? It listed methods up to `fail_stale_jobs`. Let me re-check ? line 216 lists `create_job, mark_job_running, set_job_progress, set_job_video_id, mark_job_done, mark_job_failed, get_job, fail_stale_jobs`. It omits `claim_due_job` and `set_job_waiting` and `seconds_until_next_due`. But again, the §2.7 description at line 216 does reference "self-scheduling EXECUTION_POLICY" in the version<3 note, so it's partially aware. The method list is incomplete but the §2.7 text already references the self-scheduling mechanism. Let me verify `mark_job_running` still exists (CODE_MAP lists it but the new flow uses `claim_due_job` which marks running atomically).

59. user (2026-06-19T15:49:09.149Z)

```
486:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
501:    def mark_job_running(self, job_id: str) -> bool:
588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
```

60. assistant (2026-06-19T15:49:20.570Z)

`mark_job_running` still exists, so line 216's method list is still valid (just not exhaustive ? missing `claim_due_job`/`set_job_waiting`, but those are additions, not contradictions). I'll leave §2.7 line 216 alone since it doesn't contradict code; it's only incomplete and already references self-scheduling.

Now let me verify the §1A diagram line 64-66 and other refactor claims. Let me check #227 (yolo_hybrid phase helpers ? CODE_MAP line 147 describes yolo_hybrid orchestration) and #230 (motion). Let me verify #231 `_build_preset` in rally_seg_eval and whether CODE_MAP references it (it doesn't reference rally_seg_eval symbols).

Let me now nail down the most defensible findings and check the README for any module-path claims. The README doc-index points to docs only, not code modules, so its module-structure claims are minimal. Let me verify the README references to `backend/eval/experiment.py` (line 6) and the EXPERIMENT_HARNESS "P1 SHIPPED" ? these are doc-status, owner-gated, out of scope.

Let me do a final verification on the `golden_real_fixtures` re-export to decide severity, and confirm `freeze_real_fixture`/`refresh_snapshot` location precisely.

61. user (2026-06-19T15:49:22.198Z)

```
321:def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
429:def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
454:def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
472:def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
550:def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
561:def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
583:def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) -> List[str]:
```

62. user (2026-06-19T15:49:22.555Z)

```
59:class RefusalError(ValueError):
157:def freeze_real_fixture(
```

63. assistant (2026-06-19T15:49:29.454Z)

Confirmed precisely:
- `golden_fixtures.py` defines: `replay_fixture`, `freeze_fixture`, `freeze_synthetic`, `refresh_snapshot`, `load_manifest`, `load_fixture`, `compare_expected` ? (these match CODE_MAP line 183)
- `golden_real_fixtures.py` (NEW module, #233) defines: `RefusalError`, `freeze_real_fixture` ? which CODE_MAP line 183 still attributes to `@backend/eval/golden_fixtures.py` as `::freeze_real_fixture`.

`golden_fixtures.py` re-exports them at the bottom (so `golden_fixtures.freeze_real_fixture`

still resolves via the re-export). This makes line 183's `::freeze_real_fixture` a moved symbol: it lives in `golden_real_fixtures.py` now, and that new module is entirely absent from the map. This is a legitimate, code-confirmed staleness from #233.

Let me confirm the re-export exists at the bottom of golden_fixtures.py so I describe it accurately.

64. user (2026-06-19T15:49:30.650Z)

```
760                             "attribution back-channel, so it never reaches the committed
provenance "
761                             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
762         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
763                         help="(--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
764         ap.set_defaults(time_origin_reset=True)
765         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
766                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
767         args = ap.parse_args(list(argv) if argv is not None else None)
768
769         fixtures_dir: Path = args.fixtures_dir
770
771         if args.freeze_synthetic:
772             slugs = freeze_synthetic(fixtures_dir)
773             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {',
'.join(slugs)}")
774             print(f"  -> {fixtures_dir}")
775             return 0
776
777         if args.refresh_snapshot:
778             if not args.slug and not args.all:
779                 ap.error("--refresh-snapshot requires --slug S or --all")
780             # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
781             # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
782             if not (args.reason and args.reason.strip()):
783                 ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
784                          "regression; record the deliberate reason and review the visible
diff (§6).")
785             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
786             print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
787                   f"  (reason: {args.reason})")
788             return 0
789
790         if args.freeze_real:
791             # The real-fixture subsystem (P1 + M2) lives in golden_real_fixtures (split
out); import it
792             # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always
has.
793             from backend.eval import golden_real_fixtures
794             missing = [n for n, v in (("--trajectory", args.trajectory),
795                                       ("--labels", args.labels),
796                                       ("--fps", args.fps),
797                                       ("--frame-width", args.frame_width)) if v is None]
798             if missing:
799                 ap.error("--freeze-real requires " + ", ".join(missing))
800             try:
801                 expected = golden_real_fixtures.freeze_real_fixture(
802                     args.label, args.trajectory, args.labels,
```

```
803                     license=args.license_ if args.license_ is not None else "",
804                     minors_free=args.minors_free, owned=args.owned,
805                     fps=args.fps, frame_width=args.frame_width,
806                     fixtures_dir=fixtures_dir, slug=args.slug,
807                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
808             except golden_real_fixtures.RefusalError as e:
809                 # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
zero.
810                 print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
811                 return 2
812             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
813                   f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
814             print("  §6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
815                   "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
816             return 0
817
818         # --check
819         return _cmd_check(fixtures_dir)
820
821
822     # ============================================================================
823     # Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures``
but
824     # is exposed here so every historical ``golden_fixtures.<name>`` import path stays
valid.
825     # ============================================================================
826
827     #: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``)
so the
828     #: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import
order ?
829     #: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER
bottom-of-
830     #: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own
names would
831     #: not yet be defined when this module's import of them runs). The lazy hook defers the
import to
832     #: first attribute access, by which point BOTH modules are fully initialized.
833     _REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
834         "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
```

65. assistant (2026-06-19T15:49:41.849Z)

Confirmed. CODE_MAP line 183 attributes `::freeze_real_fixture` to `@backend/eval/golden_fixtures.py`, but per #233 the function (and `RefusalError`, `_scan_forbidden`) actually live in the NEW module `golden_real_fixtures.py` and are only re-exported (lazily, PEP 562) from `golden_fixtures.py`. The new module is entirely absent from CODE_MAP §2.3. This is a genuine moved-symbol staleness.

Now I have two strong, code-confirmed findings. Let me also double-check that the §1A pipeline diagram and the line 183's broader claims (e.g. `freeze_synthetic`, `refresh_snapshot`) are still correct ? they are (confirmed above). And the `tests/test_golden_real_fixtures.py` reference in line 183 is correct (file exists).

Let me make a final assessment of the `_run_job` finding's exact replacement text. The current accurate flow: `process_video` ? `db.create_job` ? `_get_executor().submit(_drain_due_jobs)`. The worker `_drain_due_jobs` claims runnable jobs via `claim_due_job` and runs each through `_run_claimed_job` (mark done/failed, or park on `JobWaiting`). These worker functions now live in `backend/api/job_worker.py` (#234), re-exported on `backend.main`.

Let me verify the `_execute_processing` claim in line 114 is still accurate (it says it's the

former sync body and that a RAISING segmenter prune-restores). It's at main.py line 125. Quick check it still matches.

66. user (2026-06-19T15:49:43.244Z)

```
125     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
126         """The full hash ? validate ? segment ? report pipeline for one video.
127
128         This is the former synchronous /api/process body, unchanged in behavior, now run on
129         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
130         the sync endpoint used to return (UI compatibility); the per-video observability
131         report is still emitted in `finally:` and grafted as response["reports"].
132
133         `progress` is an optional callable(stage: str) used to surface coarse job progress.
134         Raises on unexpected internal errors ? the caller records those as a job failure
135         (after this function has already restored the pre-run segments, see below).
136         """
137         notify = progress or (lambda stage: None)
138
139         notify("hashing")
```

67. assistant (2026-06-19T15:49:54.962Z)

`_execute_processing` is accurate (still in main.py, former sync body). So the only stale part of line 114 is the `_run_job` worker description.

I now have my two highest-confidence, code-confirmed findings within the slice (CODE_MAP module-structure/symbol staleness vs current code from refactors). Let me craft surgical edits.

For finding 1 (line 114, the `_run_job` flow): I'll replace the two stale sub-clauses. Current stale text: `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback).

Correct: `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202 + job_id**. `::_drain_due_jobs` / `::_run_claimed_job` (worker loop in `@backend/api/job_worker.py` #234, re-exported on `backend.main`): claim runnable jobs via `db.claim_due_job` -> `_execute_processing` -> mark done/failed; a `JobWaiting` parks the job (`set_job_waiting`) for resume.

For finding 2 (line 183): change `::freeze_real_fixture` to note it lives in the new `golden_real_fixtures.py` module (re-exported).

Let me make these edits precise by using exact verbatim substrings. I'll return them via StructuredOutput (the script applies them). Let me craft minimal stale_text snippets that are unique.

Both findings are well-grounded. Let me return them.

68. user (2026-06-19T15:50:04.310Z)

Structured output provided successfully

69. assistant (2026-06-19T15:50:18.806Z)

I've completed the audit of my slice (`docs/CODE_MAP.md` + `docs/README.md` module-structure staleness vs current code) and returned my findings via StructuredOutput.

Summary of the two code-confirmed staleness items found:

1. **`docs/CODE_MAP.md` §2.0 line 114 ? `_run_job` is a dead symbol (high).** CODE_MAP describes the async `/api/process` flow as `db.create_job -> _get_executor().submit(_run_job,...)` with a `::_run_job` worker. That symbol no longer exists. `backend/main.py::process_video` (lines 264-267) submits `_drain_due_jobs`; the worker loop claims jobs via `db.claim_due_job` and runs

each through `_run_claimed_job`, both now living in `backend/api/job_worker.py` (#234) and re-exported on `backend.main`. `_run_job` appears only in docs and a test name.

2. **`docs/CODE_MAP.md` §2.3 line 183 ? `freeze_real_fixture` moved out (medium).** Per #233, `freeze_real_fixture` and `RefusalError` were split out into the new module `backend/eval/golden_real_fixtures.py` (defined at lines 59 and 157) and are only re-exported lazily from `golden_fixtures.py`. CODE_MAP still attributes `::freeze_real_fixture` to `golden_fixtures.py`, and the new `golden_real_fixtures.py` module is entirely absent from the §2.3 map.

Notes on things I checked but did NOT flag (to keep findings to verified contradictions only):
- The jobs `status` enum in line 114 / line 242 / the `get_job` data contract omits the new `waiting` state ? but `backend/main.py::get_job`'s own docstring (line 277) also omits it, so it's an in-code-consistent incompleteness rather than a doc-vs-code contradiction.
- `StitchingConfig` field enumeration (line 130) omits the #220-added `min_rally_duration` ? an incomplete enumeration, not a contradiction.
- Many `backend/eval/` modules (e.g. `ablation_pdf.py` #226, `rally_seg_eval.py`, `gt_loader.py`, `per_user_*`, `served_gate_a.py`) are absent from §2.3, but CODE_MAP is a curated navigation map (its regenerate note and §2 framing do not claim file-level exhaustiveness), so I did not treat these omissions as staleness.
- `backend/api/uploads.py` (#228) and `backend/api/job_worker.py` (#234) ? uploads is already documented at line 115; job_worker's existence is the basis of finding #1.

Relevant absolute paths: `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\job_worker.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\golden_real_fixtures.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\golden_fixtures.py`.